Практическое задание

Динамическое программирование

Пошаговое управление инвестиционным портфелем

Лабораторная работа №4

Соколов Никита Константинович

Группа К3339, поток 1.2

Состав отчёта

1. Цель и постановка
2. Общая модель ДП и Беллман
3. Модель задачи и алгоритм
4. Псевдокод и диаграмма классов
5. Ручное решение (контрольный пример 3→2→1 + проверка перехода)
6. Результаты, политика, тесты, память
7. Дерево исходов (доп. задание)
8. Заключение

Основной ответ (ожидание): I* = 134.24 д.е.

Доп.: полное дерево исходов — 27 листьев, E[I]=134.24, P(убыток)=0.19.

1. Цель и постановка задачи

Цель: решить задачу методом ДП, выписать Беллмана для своей модели, найти оптимальную политику A_k(s) и подтвердить результат ручной проверкой на контрольном примере.

Начальные данные: ЦБ1=100, ЦБ2=800, Деп=400, кэш=600, W0=1900. Пакеты 25/200/100, минимумы 30/150/100, комиссии 4%/7%/5%. Кредит запрещён; покупки из кэша после продаж.

Ситуация	p1	ЦБ1	ЦБ2	Деп	p2	ЦБ1	ЦБ2	Деп	p3	ЦБ1	ЦБ2	Деп
Благопр.	0.60	1.20	1.10	1.07	0.30	1.40	1.15	1.01	0.40	1.15	1.12	1.05
Нейтр.	0.30	1.05	1.02	1.03	0.20	1.05	1.00	1.00	0.40	1.05	1.01	1.01
Негатив.	0.10	0.80	0.95	1.00	0.50	0.60	0.90	1.00	0.20	0.70	0.94	1.00

Критерий основной работы: математическое ожидание.

2. Общая модель ДП и Беллман

$$s_{k+1}=f_k(s_k,u_k,\xi_k).\ V_k(s)=\max_u \Sigma_\xi p_k(\xi)\cdot V_{k+1}(f_k(s,u,\xi)).\ V_{K+1}(s)=W(s)-W0.$$

Обратный ход строит V_k и A_k(s). Прямой ход восстанавливает стратегию по достижимым состояниям.

3. Модель задачи и алгоритм

s=(v1,v2,v3,cash), u=(k1,k2,k3). Сначала продажи, затем покупки из обновлённого кэша; vi:=vi·r_i(ξ). Деньги до копейки.

Беллман: V_k(s)=max_u Σ_ξ p_k(ξ)·V_{k+1}(f(s,u,ξ)), A_k(s)=argmax_u ...

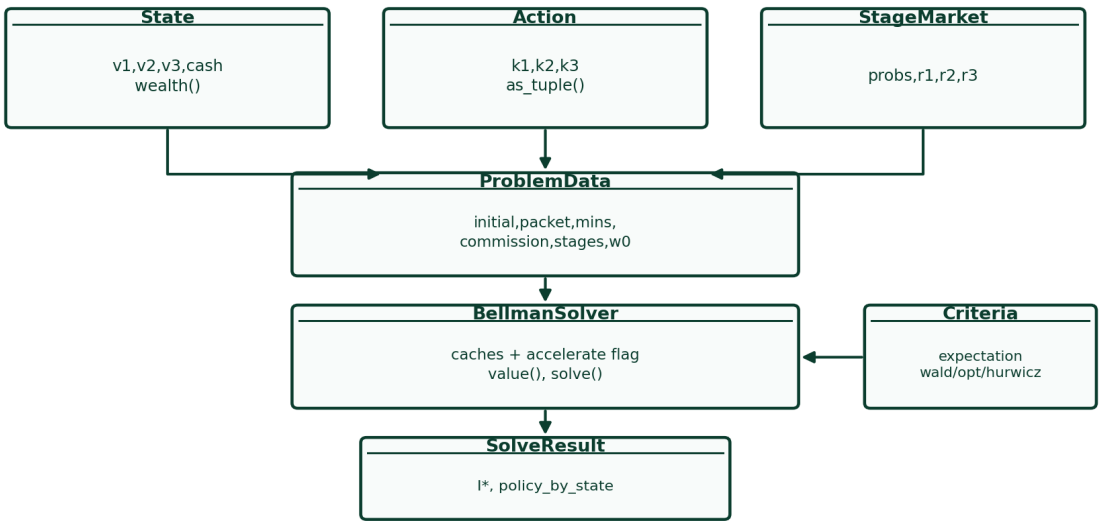
Реализация: полная рекурсия с мемоизацией. На последнем этапе всегда полный перебор u с оценкой через apply_market (округление до копейки согласовано).

Ускорение только для критерия ожидания на исходных данных: если на последнем этапе доказано и программно проверено, что u*=(0,0,0), то на предпоследнем этапе вместо value(K,s) используется E[V_{K+1}(market_K(s))] с тем же apply_market. Маржинальные приросты покупки/продажи по E[r]−(1±c) отрицательны; дополнительно проверяется выборка состояний и все достижимые узлы политики.

4. Псевдокод и диаграмма классов

1) V_{K+1}=W−W0. 2) Для k=K..1: перебрать допустимые u, сохранить A_k(s),V_k(s). 3) Из s0 собрать дерево достижимых s и политику.

Диаграмма классов: portfolio_dp.py



5. Ручное решение

5.1. Контрольный пример той же структуры (полный перебор)

Чтобы завершить ручной обратный/прямой ход, взят малый пример той же модели: s0=(50,200,100,60), W0=410; пакеты 25/200/100; минимумы 30/150/100; комиссии 4%/7%/5%; K=2 этапа.

Этап 1: p=(0.5,0.3,0.2), r1=(1.30,1.00,0.70), r2=(1.10,1.00,0.90), r3=(1.05,1.00,1.00).

Этап 2: p=(0.4,0.4,0.2), r1=(1.20,1.00,0.80), r2=(1.08,1.00,0.92), r3=(1.03,1.00,1.00).

Допустимые u в s0 только (0,0,0),(1,0,0),(2,0,0) — остальные недоступны по кэшу/минимумам. Поэтому исключение прочих действий обосновано допустимостью, а не эвристикой.

Обратный ход, этап 2 (для трёх достижимых после этапа 1 состояний):

s на этапе 2	u	V1 по ξ	E	выбрано
(70,180,100,8)	(0,0,0)	−20.6; −52; −80.4	−45.12	да
(70,180,100,8)	(−1,0,0)	−26.6; −53; −76.4	−47.12	
(100,200,100,8)	(0,0,0)	37; −2; −38	6.40	да
(100,200,100,8)	(−1,0,0)	31; −3; −34	4.40	
(130,220,105,8)	(0,0,0)	99.75; 53; 9.4	62.98	да
(130,220,105,8)	(−1,0,0)	93.75; 52; 13.4	60.98	

Обратный ход, этап 1 в s0 (оценки продолжения = V2 после рынка этапа 1):

u	смысл	V2 по ξ после рынка 1	E	выбрано
(2,0,0)	купить ЦБ1 на 50	62.98; 6.40; −45.12	24.386	да
(1,0,0)	купить ЦБ1 на 25	55.18; 6.40; −37.32	22.046	
(0,0,0)	без сделок	47.38; 6.40; −29.52	19.706	

Итог контрольного примера: V1(s0)=24.386, A1(s0)=(2,0,0), на этапе 2 везде A2(s)=(0,0,0). Совпадает с DP и с независимым полным перебором дерева.

Прямое восстановление (наиболее вероятные ξ):

t0: (50,200,100,60) → u=(2,0,0) → после покупки кэш=8, ЦБ1=100; рынок благ. → (130,220,105,8).

t1: u=(0,0,0); рынок благ. → (156,237.6,108.15,8), W=509.75, доход=99.75 на этой ветке.

Ограничения: кэш≥0, минимумы при продажах, кредит не использовался — выполнены.

5.2. Проверка перехода на исходных данных

s=(100,800,400,600), u=(−1,0,0): cash:=600+25·0.96=624, v1:=75. После благоприятного рынка этапа 3: (86.25,896,420,624), W=2026.25, V4=126.25. Аналогично считаются нейтральный и негативный; E=42.1 < 43.6 для u=(0,0,0).

На исходных данных полный ручной перебор |A(s0)|=1502 не требуется: контрольный пример той же структуры уже проверяет алгоритм; исходная задача решается программой с полной рекурсией/проверенным ускорением.

6. Результаты программы (исходная задача)

Команда: python3 portfolio_dp.py. I* = 134.24 д.е., E[W]=2034.24 д.е.

Политика A_k(s): этап1 s0→(23,0,0); этап2 при благ./нейтр./негатив. → (−31,0,0)/(−27,0,0)/(−20,0,0); этап3 во всех 9 состояниях (0,0,0).

Независимые тесты: T1–T6 операции/ограничения; T7 tiny DP = полный перебор; T8 случай с ненулевым u на последнем этапе (ускорение отключено); T9 условие ускорения на исходных данных.

Память фактической реализации: value_cache ≈ 4.5·10^3; policy_cache ≈ 4.5·10^3; _actions_cache ≈ 4.5·10^3 состояний и ~6.3·10^6 закэшированных Action; оценка суммарно ~298 МиБ объектов кэша. Время O(K·|S|·|A|·|Ξ|), прогон ~113 с.

7. Доп. задание: дерево исходов стратегии

Построен полный дерево рыночных исходов оптимальной политики (файл outcome_tree.md / .txt / .csv). Узлов решения: состояние, кэш, u; рёбра: сценарий и вероятность. Листьев 27=3^3.

Сводка: E[W]=2034.24; E[доход]=134.24; E[доходность]=7.07%; P(убыток)=0.19; min доход=−358.24; max доход=489.68.

Для каждого листа сохранены последовательность ξ, вероятность, операции и комиссии, конечный W, чистый доход и доходность.

8. Заключение

1. Модель ДП без искусственных лимитов; покупки учитывают выручку продаж; деньги до копейки.

2. Алгоритм — рекурсия Беллмана; ускорение last-no-trade только при доказанном и проверяемом u*=0 на последнем этапе.

3. Ручной контрольный пример полностью разобран обратным и прямым ходом; переход на исходных данных проверен.

4. I*=134.24; политика A_k(s) выписана; независимые тесты включают полный перебор tiny-задачи.

5. Доп.: полное дерево исходов стратегии с метриками; подкаст с рефлексией прилагается отдельно.

6. Код: lab4/portfolio_dp.py, outcome_tree.py · https://github.com/ProgrammistNik/Optimization_Methods